LLMs and Formal Methods: A survey

1 April 2026

Contents

1 Scope and structure **1**

2 Taxonomy **2**

3 Surveys and roadmaps **2**

4 Specification languages and requirements **2**

5 Verified code generation and proof synthesis for code **3**

6 AI-assisted mechanised proving **4**

7 Temporal logic, TLA+, model checking, and hardware formal verification **5**

8 Benchmarks and datasets **5**

9 Adjacent specification and AI-native coding systems **6**

Complete bibliography **6**

1 Scope and structure

This survey is organised around the two-way intersection between large language models and formal methods: *formal methods for constraining or checking AI-assisted software development*, and *LLMs for lowering the cost of formal specification, verification, and mechanised proof*. After a short section of surveys and roadmaps, the material is organised directly by technical category, mixing papers, systems, benchmarks, and industry artefacts where that gives the clearest picture of the space. The main emphasis is on systems with a direct formal-methods component, but the survey also notes adjacent specification and programming artefacts when they illuminate likely interfaces between natural-language intent, intermediate specifications, AI-native implementation languages, and verifier-oriented workflows. Representative surveys and roadmaps that frame the field include the main autoformalisation survey, the software-requirements survey, the theorem-proving survey, and broader roadmaps on trustworthy AI agents and formal mathematical reasoning. [1, 2, 3, 4, 5, 6]

2 Taxonomy

Area	What sits here	Section
Surveys and roadmaps	High-level surveys, position papers, and roadmaps that frame the field and its major research directions.	§3
Specification languages and requirements	Natural language to logic/specification, intent formalisation, and auditable requirement capture.	§4
Verified code generation and proof synthesis for code	Verifier-in-the-loop generation of code, contracts, assertions, and proofs for Dafny, Verus, ACSL, SPARK, and related systems.	§5
AI-assisted mechanised proving	Interactive theorem proving, premise selection, tactic generation, whole-proof synthesis, and proof repair in Lean, Isabelle, Rocq/Coq, and Metamath.	§6
Temporal logic, TLA+, model checking, and hardware FV	Temporal-logic generation, TLAPS/TLA+ proof support, and assertion synthesis for hardware formal verification.	§7
Benchmarks and datasets	Shared evaluation sets for theorem proving, verifiable code generation, repository-scale proof, TLA+, and hardware FV.	§8
Adjacent specification and AI-native coding systems	Spec-centric languages, AI-oriented implementation languages, and spec-driven development toolchains that are not themselves formal methods results but may shape how intent is structured before verification.	§9

3 Surveys and roadmaps

- **Autoformalisation survey**. [1]
- **Survey on LLMs for software-requirement formalisation**. [2]
- **Survey on deep learning for theorem proving**. [3]
- **Roadmap on LLMs and formal methods for trustworthy agents**. [4]
- **Formal mathematical reasoning as an AI frontier**. [5]
- **Two-way roadmap between requirements engineering and LLMs**. [6]

4 Specification languages and requirements

- **VERIFAI**. Early work on traceability and verification of natural-language requirements, useful as background for NL-to-spec workflows. [7]

- **Formal Specifications from Natural Language**. Direct NL-to-formal-spec framing paper. [8]
- **Autoformalization with Large Language Models**. Early LLM-based autoformalisation of informal mathematical text. [9]
- **nl2spec**. Interactive translation from unstructured requirements to temporal logic with editable sub-translations. [10]
- **NL2TL**. A general framework for mapping natural-language instructions to temporal logic across multiple domains. [11]
- **SpecGen**. Automated generation of formal specifications from natural-language requirements. [12]
- **Specify What?** and **Seeking Specifications**. Neuro-symbolic work on deriving and refining specifications from informal descriptions. [13, 14]
- **Doc2Spec**. A multi-agent approach that induces formal specifications from natural-language documents. [15]
- **Trustworthy Formal Natural Language Specifications**. A proof-assistant-native route to formalising natural-language specifications inside Lean. [16]
- **Microsoft intent formalisation agenda**. Industry framing that treats user-intent formalisation as a central reliability problem for AI-assisted coding. [17]
- **nl2postcond** and follow-on evaluation work. Microsoft-led efforts on turning programmer intent into checkable postconditions and evaluating this route for verification-aware languages. [18, 19]
- **Supporting Software Formal Verification with Large Language Models: An Experimental Study**. Broader evidence on where LLM assistance helps in formal-specification and verification workflows. [20]

5 Verified code generation and proof synthesis for code

- **DafnySynth / dafny-annotator**. LLM-assisted synthesis of Dafny assertions, invariants, and proof-oriented annotations. [21]
- **Laurel**. Uses LLMs to unblock automated verification by synthesising useful proof and specification steps. [22]
- **Verification with Ada/SPARK**. Evaluation of LLM-generated code in a SPARK-based verification setting. [23]
- **LLM code generation with formal specifications and reactive synthesis**. Couples code generation with formal specifications and synthesis constraints. [24]
- **VeCoGen**. Formally verified C generation using LLMs plus a verifier-guided refinement loop. [25]
- **Automatic generation of formal specification and verification annotations using LLMs and test oracles**. Annotation-generation workflow for verification tools. [26]
- **Evaluating LLM-generated ACSL annotations**. Assessment of annotation quality for Frama-C-style verification workflows. [27]

- **AlphaVerus**. Self-improving verified code generation for Verus/Rust. [28]
- **RAG-Verus**. Repository-level proof synthesis for Verus with retrieval augmentation and repository context. [29]
- **Microsoft verus-proof-synthesis**. Umbrella repository for Microsoft's Verus verification automation stack. [30]
- **AutoVerus**. Automated proof generation for Rust/Verus. [31]
- **VeruSAGE**. Agent-based verification for repository-scale Rust systems. [32]
- **VeriStruct**. Planner-based verification of Verus data-structure modules. [33]
- **Generating Safety-Critical Automotive C-programs using LLMs with Formal Verification**. Joint industry-academic evaluation in a safety-critical domain. [34]

6 AI-assisted mechanised proving

- **GPT-f**. Early neural theorem-proving baseline for proof search guidance. [35]
- **Draft, Sketch, and Prove**. Guides formal theorem provers using informal proof sketches. [36]
- **ProofNet**. Autoformalisation and formal proving for undergraduate-level mathematics. [37]
- **Baldur**. Whole-proof generation and repair with large language models. [38]
- **LeanDojo / ReProver**. Open tooling and datasets for Lean-based proving with retrieval-augmented models. [39]
- **COPRA**. In-context agentic theorem proving for interactive assistants. [40]
- **DeepSeek-Prover**. Open Lean prover line with synthetic-data training; the public repository tracks released models. [41, 42]
- **SubgoalXL**. Subgoal-based expert learning for theorem proving. [43]
- **LEGO-Prover**. Neural theorem proving with growing libraries. [44]
- **Lean Copilot**. LLM inference inside Lean for tactic suggestion, proof search, and premise selection. [45]
- **APOLLO**. Lean+LLM collaboration driven by compiler-guided repair. [46]
- **HybridProver**. Combines theorem proving with LLM-driven proof synthesis and repair. [47]
- **A Minimalist Proof Language for Neural Theorem Proving over Isabelle/HOL**. Proof-language design for neural theorem proving over Isabelle/HOL. [48]
- **Agentic Program Verification / AutoRocq**. Agentic Rocq/Coq verification via iterative prover feedback. [49]
- **AutoReal on seL4**. LLM-driven theorem proving in an industrial-scale verification setting. [50]
- **Vibe Coding an LLM-powered Theorem Prover / Isabellm**. Lightweight Isabelle/HOL proving with both a paper and an open implementation. [51, 52]

- **LeanNavigator**. Large-scale generation of additional Lean theorems and proofs from Mathlib4. [53]
- **AlphaProof programme**. Google DeepMind's formal mathematical reasoning effort, framed publicly through reinforcement learning and formal proof search. [54, 55]
- **Harmonic / Aristotle**. Industry effort on olympiad-level automated theorem proving and formal mathematical reasoning. [56]
- **formal-conjectures**. Repository of Lean-formalised conjectures intended to support formal reasoning research. [57]

7 Temporal logic, TLA+, model checking, and hardware formal verification

- **Guiding LLM Temporal Logic Generation with Explicit Separation of Data and Control**. Temporal-logic generation from natural-language inputs with structural guidance. [58]
- **Retrieval-Augmented TLAPS Proof Generation with Large Language Models**. Proof generation for TLAPS using retrieved verified proof fragments. [59]
- **Towards Language Model Guided TLA+ Proof Automation**. Hierarchical decomposition for TLA+ proof obligations. [60]
- **AssertLLM**. Assertion generation from design specifications and waveforms for hardware verification. [61]
- **STELLAR**. Structure-guided retrieval and generation for SystemVerilog assertions. [62]

8 Benchmarks and datasets

- **miniF2F**, **PUTNAMBENCH**, and **FormalMATH**. Benchmarks for formal mathematical reasoning and theorem proving across systems and domains. [63, 64, 65]
- **FIMO**. Challenge dataset for automated theorem proving. [66]
- **CLEVER** and **VERINA**. Benchmarks targeting verifier-in-the-loop code generation and proof-oriented programming tasks. [67, 68]
- **A benchmark for vericoding**, **AlgoVeri**, and **CASP**. Evaluation sets for formally verified program synthesis and code-level proof generation. [69, 70, 71]
- **DafnyBench** and **DafnyCOMP**. Benchmarks and competitions for LLM-assisted Dafny verification workflows. [72, 73]
- **VeriSoftBench**. Repository-scale Lean benchmark focused on software-verification obligations rather than pure mathematics. [74]
- **TLAiBench**. Benchmark repository for TLA+ specification tasks. [75]
- **AssertionBench** and **FVEval**. Benchmarks for hardware assertion generation and hardware formal-verification evaluation. [76, 77]

9 Adjacent specification and AI-native coding systems

- **Mog**. An AI-agent-oriented embedded language designed to be written by LLMs, with an explicit capability model and a compact language specification. [78]
- **CodeSpeak**. A spec-centric programming language and toolchain that generates conventional code from maintained specifications using LLMs. [79]
- **Spec Kit**. GitHub's open-source toolkit for spec-driven development with AI agents, centred on turning specifications into implementation plans and executable workflows. [80]

Complete bibliography

[1] Ke Weng et al. "Autoformalization in the Era of Large Language Models: A Survey". In: *CoRR* abs/2505.23486 (2025). URL: https://doi.org/10.48550/arXiv.2505.23486 [local PDF].

[2] Arshad Beg, Diarmuid P. O'Donoghue, and Rosemary Monahan. "A Short Survey on Formalising Software Requirements using Large Language Models". In: *CoRR* abs/2506.11874 (2025). URL: https://doi.org/10.48550/arXiv.2506.11874 [local PDF].

[3] Zhaoyu Li et al. "A Survey on Deep Learning for Theorem Proving". In: *CoRR* abs/2404.09939 (2024). URL: https://doi.org/10.48550/arXiv.2404.09939 [local PDF].

[4] Yedi Zhang et al. "The Fusion of Large Language Models and Formal Methods for Trustworthy AI Agents: A Roadmap". In: *CoRR* abs/2412.06512 (2024). URL: https://doi.org/10.48550/arXiv.2412.06512 [local PDF].

[5] Kaiyu Yang et al. "Position: Formal Mathematical Reasoning - A New Frontier in AI". In: *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025 - Position Paper Track*. OpenReview.net, 2025. URL: https://proceedings.mlr.press/v267/yang25az.html [local PDF].

[6] Alessio Ferrari and Paola Spoletini. "Formal requirements engineering and large language models: A two-way roadmap". In: *Inf. Softw. Technol.* 181 (2025), p. 107697. URL: https://doi.org/10.1016/j.infsof.2025.107697 [local PDF].

[7] *VERIFAI*. Project page. 2024. URL: https://www.cs.nuim.ie/research/pop/verifai.html [local PDF].

[8] Christopher Hahn et al. "Formal Specifications from Natural Language". In: *CoRR* abs/2206.01962 (2022). URL: https://doi.org/10.48550/arXiv.2206.01962 [local PDF].

[9] Yuhuai Wu et al. "Autoformalization with Large Language Models". In: *CoRR* abs/2205.12615 (2022). URL: https://doi.org/10.48550/arXiv.2205.12615 [local PDF].

[10] Matthias Cosler et al. *nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models*. 2023. URL: https://arxiv.org/abs/2303.04864 [local PDF].

[11] Yongchao Chen et al. "NL2TL: Transforming Natural Languages to Temporal Logics using Large Language Models". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*. Ed. by Houda Bouamor, Juan Pino, and Kalika Bali. Association for Computational Linguistics, 2023, pp. 15880–15903. URL: https://doi.org/10.18653/v1/2023.emnlp-main.985 [local PDF].

[12] Lezhi Ma et al. *SpecGen: Automated Generation of Formal Program Specifications via Large Language Models*. 2025. URL: https://arxiv.org/abs/2401.08807 [local PDF].

[13] George Granberry, Wolfgang Ahrendt, and Moa Johansson. *Specify What? Enhancing Neural Specification Synthesis by Symbolic Methods*. 2024. URL: https://arxiv.org/abs/2406.15540 [local PDF].

[14] George Granberry, Wolfgang Ahrendt, and Moa Johansson. *Seeking Specifications: The Case for Neuro-Symbolic Specification Synthesis*. 2025. URL: https://arxiv.org/abs/2504.21061 [local PDF].

[15] Shihao Xia et al. *Doc2Spec: Synthesizing Formal Programming Specifications from Natural Language via Grammar Induction*. 2026. URL: https://arxiv.org/abs/2602.04892 [local PDF].

[16] Colin S. Gordon and Sergey Matskevich. *Trustworthy Formal Natural Language Specifications*. 2023. URL: https://arxiv.org/abs/2310.03885 [local PDF].

[17] *Intent Formalization: A Grand Challenge for Reliable Coding in the Age of AI Agents*. Project / position page. 2025. URL: https://www.microsoft.com/en-us/research/publication/intent-formalization-a-grand-challenge-for-reliable-coding-in-the-age-of-ai-agents/ [local PDF].

[18] Madeline Endres et al. *Can Large Language Models Transform Natural Language Intent into Formal Method Postconditions?* 2024. URL: https://arxiv.org/abs/2310.01831 [local PDF].

[19] Shuvendu K. Lahiri. *Evaluating LLM-driven User-Intent Formalization for Verification-Aware Languages*. 2024. URL: https://arxiv.org/abs/2406.09757 [local PDF].

[20] Weiqi Wang et al. *Supporting Software Formal Verification with Large Language Models: An Experimental Study*. 2025. URL: https://arxiv.org/abs/2507.04857 [local PDF].

[21] Gabriel Poesia, Chloe Loughridge, and Nada Amin. "dafny-annotator: AI-Assisted Verification of Dafny Programs". In: *CoRR* abs/2411.15143 (2024). URL: https://doi.org/10.48550/arXiv.2411.15143 [local PDF].

[22] Eric Mugnier et al. "Laurel: Unblocking Automated Verification with Large Language Models". In: *Proc. ACM Program. Lang.* 9.OOPSLA1 (2025), pp. 1519–1545. URL: https://doi.org/10.1145/3720499 [local PDF].

[23] Marcos Cramer and Lucian McIntyre. "Verifying LLM-Generated Code in the Context of Software Verification with Ada/SPARK". In: *Proceedings of the 20th International Conference on Software Technologies, ICSOFT 2025, Bilbao, Spain, June 10-12, 2025*. Ed. by Massimo Mecella, Arend Rensink, and Leszek A. Maciaszek. SCITEPRESS, 2025, pp. 39–50. URL: https://doi.org/10.5220/0013461900003964 [local PDF].

[24] William Murphy et al. "Combining LLM Code Generation with Formal Specifications and Reactive Program Synthesis". In: *CoRR* abs/2410.19736 (2024). URL: https://doi.org/10.48550/arXiv.2410.19736 [local PDF].

[25] Merlijn Sevenhuijsen, Khashayar Etemadi, and Mattias Nyberg. "VeCoGen: Automating Generation of Formally Verified C Code With Large Language Models". In: *13th IEEE/ACM International Conference on Formal Methods in Software Engineering, FormaliSE@ICSE 2025, Ottawa, ON, Canada, April 27-28, 2025*. IEEE, 2025, pp. 101–112. URL: https://doi.org/10.1109/FormaliSE66629.2025.00017 [local PDF].

[26] João Pascoal Faria et al. "Automatic Generation of Formal Specification and Verification Annotations Using LLMs and Test Oracles". In: *CoRR* abs/2601.12845 (2026). URL: https://doi.org/10.48550/arXiv.2601.12845 [local PDF].

[27] Arshad Beg, Diarmuid P. O'Donoghue, and Rosemary Monahan. "Evaluating LLM-Generated ACSL Annotations for Formal Verification". In: *CoRR* abs/2602.13851 (2026). URL: https://doi.org/10.48550/arXiv.2602.13851 [local PDF].

[28] *AlphaVerus*. Project page. 2025. URL: https://alphaverus.github.io/ [local PDF].

[29] Sicheng Zhong et al. "RAG-Verus: Repository-Level Program Verification with LLMs using Retrieval Augmented Generation". In: *CoRR* abs/2502.05344 (2025). URL: https://doi.org/10.48550/arXiv.2502.05344 [local PDF].

[30] *Microsoft verus-proof-synthesis*. Repository / project page. 2025. URL: https://github.com/microsoft/verus-proof-synthesis [local PDF].

[31] Chenyuan Yang et al. "AutoVerus: Automated Proof Generation for Rust Code". In: *Proc. ACM Program. Lang.* 9.OOPSLA2 (2025), pp. 3454–3482. URL: https://doi.org/10.1145/3763174 [local PDF].

[32] Chenyuan Yang et al. "VeruSAGE: A Study of Agent-Based Verification for Rust Systems". In: *CoRR* abs/2512.18436 (2025). URL: https://doi.org/10.48550/arXiv.2512.18436 [local PDF].

[33] Chuyue Sun et al. "VeriStruct: AI-assisted Automated Verification of Data-Structure Modules in Verus". In: *CoRR* abs/2510.25015 (2025). URL: https://doi.org/10.48550/arXiv.2510.25015 [local PDF].

[34] *Generating Safety-Critical Automotive C-programs using LLMs with Formal Verification*. Joint industry-academic paper. 2025. URL: https://proceedings.mlr.press/v284/sevenhuijsen25a.html [local PDF].

[35] Stanislas Polu and Ilya Sutskever. "Generative Language Modeling for Automated Theorem Proving". In: *CoRR* abs/2009.03393 (2020). URL: https://arxiv.org/abs/2009.03393 [local PDF].

[36] Albert Q. Jiang et al. "Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs". In: *CoRR* abs/2210.12283 (2022). URL: https://doi.org/10.48550/arXiv.2210.12283 [local PDF].

[37] Zhangir Azerbayev et al. "ProofNet: Autoformalizing and Formally Proving Undergraduate-Level Mathematics". In: *CoRR* abs/2302.12433 (2023). URL: https://doi.org/10.48550/arXiv.2302.12433 [local PDF].

[38] Emily First et al. "Baldur: Whole-Proof Generation and Repair with Large Language Models". In: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023*. Ed. by Satish Chandra, Kelly Blincoe, and Paolo Tonella. ACM, 2023, pp. 1229–1241. URL: https://doi.org/10.1145/3611643.3616243 [local PDF].

[39] Kaiyu Yang et al. "LeanDojo: Theorem Proving with Retrieval-Augmented Language Models". In: *CoRR* abs/2306.15626 (2023). URL: https://doi.org/10.48550/arXiv.2306.15626 [local PDF].

[40] *An In-Context Learning Agent for Formal Theorem-Proving*. Research paper / agent. 2024. URL: https://openreview.net/forum?id=V7HRrxXUhN [local PDF].

[41] Huajian Xin et al. "DeepSeek-Prover: Advancing Theorem Proving in LLMs through Large-Scale Synthetic Data". In: *CoRR* abs/2405.14333 (2024). URL: https://doi.org/10.48550/arXiv.2405.14333 [local PDF].

[42] *DeepSeek-Prover-V1.5*. Repository / project page. 2025. URL: https://github.com/deepseek-ai/DeepSeek-Prover-V1.5 [local PDF].

[43] Xueliang Zhao et al. "SubgoalXL: Subgoal-based Expert Learning for Theorem Proving". In: *CoRR* abs/2408.11172 (2024). URL: https://doi.org/10.48550/arXiv.2408.11172 [local PDF].

[44] Haiming Wang et al. "LEGO-Prover: Neural Theorem Proving with Growing Libraries". In: *CoRR* abs/2310.00656 (2023). URL: https://doi.org/10.48550/arXiv.2310.00656 [local PDF].

[45] *Lean Copilot: Large Language Models as Copilots for Theorem Proving in Lean*. Conference paper. 2025. URL: https://proceedings.mlr.press/v288/song25a.html [local PDF].

[46] *APOLLO: Automated LLM and Lean Collaboration for Theorem Proving*. Research paper / system. 2025. URL: https://openreview.net/forum?id=fxDCgOruk0 [local PDF].

[47] Jilin Hu et al. "HybridProver: Augmenting Theorem Proving with LLM-Driven Proof Synthesis and Refinement". In: *CoRR* abs/2505.15740 (2025). URL: https://doi.org/10.48550/arXiv.2505.15740 [local PDF].

[48] Qiyuan Xu et al. *A Minimalist Proof Language for Neural Theorem Proving over Isabelle/HOL*. 2025. URL: https://arxiv.org/abs/2507.18885 [local PDF].

[49] Haoxin Tu et al. *Agentic Verification of Software Systems*. 2026. URL: https://arxiv.org/abs/2511.17330 [local PDF].

[50] Jianyu Zhang et al. "Towards Real-World Industrial-Scale Verification: LLM-Driven Theorem Proving on seL4". In: *CoRR* abs/2602.08384 (2026). URL: https://doi.org/10.48550/arXiv.2602.08384 [local PDF].

[51] Zhe Hou. *Vibe Coding an LLM-powered Theorem Prover*. 2026. URL: https://arxiv.org/abs/2601.04653 [local PDF].

[52] *Isabellm: A Playground for Free and Lightweight LLM-Guided Isabelle/HOL Provers*. Repository / project page. 2026. URL: https://github.com/zhehou/llm-isabelle [local PDF].

[53] David Yin and Jing Gao. "Generating Millions Of Lean Theorems With Proofs By Exploring State Transition Graphs". In: *CoRR* abs/2503.04772 (2025). URL: https://doi.org/10.48550/arXiv.2503.04772 [local PDF].

[54] *AI solves IMO problems at silver-medal level*. Google DeepMind blog post. 2024. URL: https://deepmind.google/blog/ai-solves-imo-problems-at-silver-medal-level/.

[55] *Olympiad-level formal mathematical reasoning with reinforcement learning*. Nature article. 2025. URL: https://www.nature.com/articles/s41586-025-09833-y.

[56] Tudor Achim et al. "Aristotle: IMO-level Automated Theorem Proving". In: *CoRR* abs/2510.01346 (2025). URL: https://doi.org/10.48550/arXiv.2510.01346 [local PDF].

[57] *Google DeepMind formal-conjectures*. Repository / project page. 2024. URL: https://github.com/google-deepmind/formal-conjectures.

[58] William Murphy et al. "Guiding LLM Temporal Logic Generation with Explicit Separation of Data and Control". In: *CoRR* abs/2406.07400 (2024). URL: https://doi.org/10.48550/arXiv.2406.07400 [local PDF].

[59] Yuhao Zhou. *Retrieval-Augmented TLAPS Proof Generation with Large Language Models*. 2025. URL: https://arxiv.org/abs/2501.03073 [local PDF].

[60] Yuhao Zhou and Stavros Tripakis. "Towards Language Model Guided TLA+ Proof Automation". In: *CoRR* abs/2512.09758 (2025). URL: https://doi.org/10.48550/arXiv.2512.09758 [local PDF].

[61] Zhiyuan Yan et al. "AssertLLM: Generating Hardware Verification Assertions from Design Specifications via Multi-LLMs". In: *Proceedings of the 30th Asia and South Pacific Design Automation Conference, ASPDAC 2025, Tokyo, Japan, January 20-23, 2025*. Ed. by Yuichi Nakamura and Yu Wang. ACM, 2025, pp. 614–621. URL: https://doi.org/10.1145/3658617.3697756 [local PDF].

[62] Saeid Rajabi, Chengmo Yang, and Satwik Patnaik. "STELLAR: Structure-guided LLM Assertion Retrieval and Generation for Formal Verification". In: *CoRR* abs/2601.19903 (2026). URL: https://doi.org/10.48550/arXiv.2601.19903 [local PDF].

[63] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. "miniF2F: a cross-system benchmark for formal Olympiad-level mathematics". In: *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL: https://openreview.net/forum?id=9ZPegFuFTFv [local PDF].

[64] George Tsoukalas et al. "PutnamBench: Evaluating Neural Theorem-Provers on the Putnam Mathematical Competition". In: *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*. Ed. by Amir Globersons et al. 2024. URL: http://papers.nips.cc/paper%5C_files/paper/2024/hash/1582eaf9e0cf349e1e5a6ee453100aa1-Abstract-Datasets%5C_and%5C_Benchmarks%5C_Track.html [local PDF].

[65] Zhouliang Yu et al. "FormalMATH: Benchmarking Formal Mathematical Reasoning of Large Language Models". In: *CoRR* abs/2505.02735 (2025). URL: https://doi.org/10.48550/arXiv.2505.02735 [local PDF].

[66] Chengwu Liu et al. "FIMO: A Challenge Formal Dataset for Automated Theorem Proving". In: *CoRR* abs/2309.04295 (2023). URL: https://doi.org/10.48550/arXiv.2309.04295 [local PDF].

[67] Amitayush Thakur et al. "CLEVER: A Curated Benchmark for Formally Verified Code Generation". In: *CoRR* abs/2505.13938 (2025). URL: https://doi.org/10.48550/arXiv.2505.13938 [local PDF].

[68] Zhe Ye et al. "VERINA: Benchmarking Verifiable Code Generation". In: *CoRR* abs/2505.23135 (2025). URL: https://doi.org/10.48550/arXiv.2505.23135 [local PDF].

[69] Sergiu Bursuc et al. "A benchmark for vericoding: formally verified program synthesis". In: *CoRR* abs/2509.22908 (2025). URL: https://doi.org/10.48550/arXiv.2509.22908 [local PDF].

[70] Haoyu Zhao et al. "AlgoVeri: An Aligned Benchmark for Verified Code Generation on Classical Algorithms". In: *CoRR* abs/2602.09464 (2026). URL: https://doi.org/10.48550/arXiv.2602.09464 [local PDF].

[71] Niclas Hertzberg et al. "CASP: An Evaluation Dataset for Formal Verification of C Code". In: *Bridging the Gap Between AI and Reality - Third International Conference on Bridging the Gap between AI and Reality, AISoLA 2025, Rhodes, Greece, November 1-5, 2025, Selected Papers*. Ed. by Bernhard Steffen. Lecture Notes in Computer Science. Springer, 2025, pp. 63–82. URL: https://doi.org/10.1007/978-3-032-07132-3%5C_5 [local PDF].

[72] Chloe Loughridge et al. "DafnyBench: A Benchmark for Formal Software Verification". In: *Trans. Mach. Learn. Res.* 2025 (2025). URL: https://openreview.net/forum?id=yBgTVWccIx [local PDF].

[73] Xu Xu et al. "Local Success Does Not Compose: Benchmarking Large Language Models for Compositional Formal Verification". In: *CoRR* abs/2509.23061 (2025). URL: https://doi.org/10.48550/arXiv.2509.23061 [local PDF].

[74] Yutong Xin et al. "VeriSoftBench: Repository-Scale Formal Verification Benchmarks for Lean". In: *CoRR* abs/2602.18307 (2026). URL: https://doi.org/10.48550/arXiv.2602.18307 [local PDF].

[75] *TLAiBench*. Benchmark repository. 2025. URL: https://github.com/tlaplus/TLAiBench.

[76] Vaishnavi Pulavarthi et al. "AssertionBench: A Benchmark to Evaluate Large-Language Models for Assertion Generation". In: *Findings of the Association for Computational Linguistics: NAACL 2025, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*. Ed. by Luis Chiruzzo, Alan Ritter, and Lu Wang. Findings of ACL. Association for Computational Linguistics, 2025, pp. 8058–8065. URL: https://doi.org/10.18653/v1/2025.findings-naacl.449 [local PDF].

[77] Minwoo Kang et al. "FVEval: Understanding Language Model Capabilities in Formal Verification of Digital Hardware". In: *CoRR* abs/2410.23299 (2024). URL: https://doi.org/10.48550/arXiv.2410.23299 [local PDF].

[78] *Mog*. Project / language specification. 2026. URL: https://moglang.org/.

[79] *CodeSpeak*. Project / language page. 2026. URL: https://codespeak.dev/.

[80] *GitHub Spec Kit*. Repository / toolkit. 2025. URL: https://github.com/github/spec-kit.